

Exercices SQL

Création de tables sous SQLite3 : Contrainte de référence

I. Création de tables

3.

```
CREATE TABLE groupes_fournisseurs (  
    id_groupe INTEGER PRIMARY KEY,  
    nom_groupe TEXT NOT NULL  
);
```

Cette commande permet d'abord de créer la table sous le nom "groupes_fournisseurs". Ensuite on crée 2 attributs(colonnes) pour cette table:

- L'attribut id_groupe est un entier(INTEGER) et une clé primaire(PRIMARY KEY).
- L'attribut nom_groupe est une chaîne de caractères(TEXT) et il ne peut pas être vide(NOT NULL).

4a.

Pour vérifier que c'est bien le cas, j'ai tapé les commandes suivantes:

```
INSERT INTO groupes_fournisseurs (id_groupe) VALUES (1);
```

Dans les deux cas ça renvoi une erreur.

Contrairement, si je tape la commande suivante, ça renvoi un code succès.

```
INSERT INTO groupes_fournisseurs (id_groupe, nom_groupe) VALUES (2, 'Caretour');
```

4b.

Sa clé primaire est "id_groupe", la clé primaire de l'exemple précédent est "1".

5.

```
CREATE TABLE fournisseurs (  
    id_fournisseur INTEGER PRIMARY KEY,  
    nom_fournisseur TEXT NOT NULL,  
    id_groupe INTEGER,  
    FOREIGN KEY (id_groupe)  
    REFERENCES groupes_fournisseurs (id_groupe)  
);
```

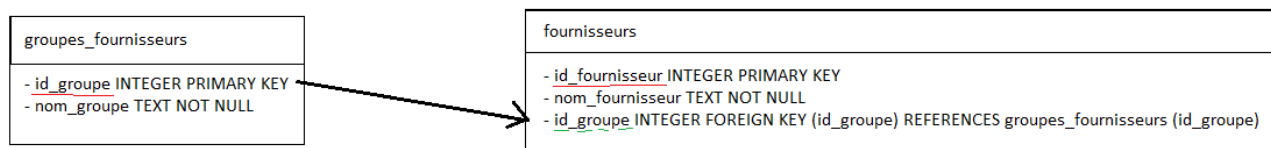
Cette commande permet d'abord de créer la table sous le nom "fournisseurs". Ensuite on crée 2 attributs(colonnes) pour cette table:

- L'attribut id_fournisseurs est un entier(INTEGER) et une clé primaire(PRIMARY KEY).
- L'attribut nom_fournisseur est une chaîne de caractères(TEXT) et il ne peut pas être vide(NOT NULL).
- L'attribut id_groupe est une clé étrangère(FOREIGN KEY).

6.

Sa clé étrangère est "id_groupe". Elle fait référence à la clé primaire "id_groupe" de la table "groupes_fournisseurs".

7.



8.



9.

```

INSERT INTO groupes_fournisseurs (id_groupe, nom_groupe)
VALUES
  (1, 'mécanique'),
  (2, 'électronique'),
  (3, 'optique');
  
```

La commande suivante permet d'insérer ces trois lignes dans la table "groupes_fournisseurs":

- On insère l'id "1" et le nom de groupe "mécanique" dans la table.
- On insère l'id "2" et le nom de groupe "électronique" dans la table.
- On insère l'id "3" et le nom de groupe "optique" dans la table.

10.

```
SELECT * FROM groupes_fournisseurs;
```

Cette commande permet d'afficher l'id et le nom de groupe de tous les éléments qui sont dans la table. Dans ce cas, cette commande affichera:

! id_groupe	nom_groupe
1	mécanique
2	électronique
3	optique

11.

```
INSERT INTO fournisseurs (id_fournisseur, nom_fournisseur, id_groupe)
VALUES
(1, 'lextronic', 2),
(2, 'gotronic', 2);
```

La commande suivante permet d'insérer ces deux lignes dans la table "fournisseurs":

- On insère l'id de fournisseur "1" et le nom de fournisseur "lextronic" dans la table. Ainsi on insère un fournisseur qui fait référence à la table *groupes_fournisseurs*, dans ce cas "2" fait référence à *électronique*.
- On insère l'id de fournisseur "2" et le nom de fournisseur "gotronic" dans la table. Ainsi on insère un fournisseur qui fait référence à la table *groupes_fournisseurs*, dans ce cas "2" fait référence à *électronique*.

12.

```
SELECT * FROM fournisseurs;
```

Cette commande renvoi la table suivante:

! id_fournisseur	nom_fournisseur	id_groupe
1	lextronic	2
2	gotronic	2

13.

```
INSERT INTO fournisseurs (id_fournisseur, nom_fournisseur, id_groupe)
VALUES (3, 'carrefour', 4);
```

Cette instruction retourne une erreur, car l'id groupe "4" n'existe pas dans la table *groupes_fournisseurs* à laquelle il fait référence.

14.

Tout d'abord je dois créer le groupe de fournisseurs 4 nommé *grandes_surfaces*:

```
INSERT INTO groupes_fournisseurs (id_groupe, nom_groupe)
VALUES
(4, 'grandes_surfaces')
```

Ensuite, on peut insérer le fournisseur **carrefour** dans le groupe de fournisseurs "4" nommé **grandes_surfaces**:

```
INSERT INTO fournisseurs (id_fournisseur, nom_fournisseur, id_groupe)
VALUES (3, 'carrefour', 4);
```

16.

```
DELETE FROM fournisseurs
WHERE id_fournisseur = 3;
```

Cette commande permet de supprimer l'élément de la liste *fournisseurs* qui a pour id de fournisseur "3". Dans ce cas on supprime carrefour.

18.

```
DELETE FROM groupes_fournisseurs
WHERE id_groupe = 2;
```

Cette instruction devrait nous permettre de supprimer l'élément de la liste *groupes_fournisseurs* qui a pour id de groupe "2". Mais ça ne marche pas car il y a des éléments dans la table *fournisseurs* qui font référence à cet id.

****19.**** Tout d'abord il faut supprimer les éléments de la table *fournisseurs* qui font référence à l'id de groupe "2":

```
DELETE FROM fournisseurs
WHERE id_groupe = 2;
```

Puis on supprime le groupe de fournisseurs "2":

```
DELETE FROM groupes_fournisseurs  
WHERE id_groupe = 2;
```

20.

```
DROP TABLE fournisseurs;
```

Cette commande permet de supprimer la table nommée *fournisseurs*

22. Pour vérifier si on peut supprimer une table contenant des données, j'ai supprimé la table *groupe_fournisseurs*, et cela a marché. Donc, on peut supprimer une table contenant des données.

```
DROP TABLE groupes_fournisseurs;
```

Manipulation de tables en SQL

I. La base de données

1.

Les tables contenues dans la base sont : *evolution* et *ville*.

2.

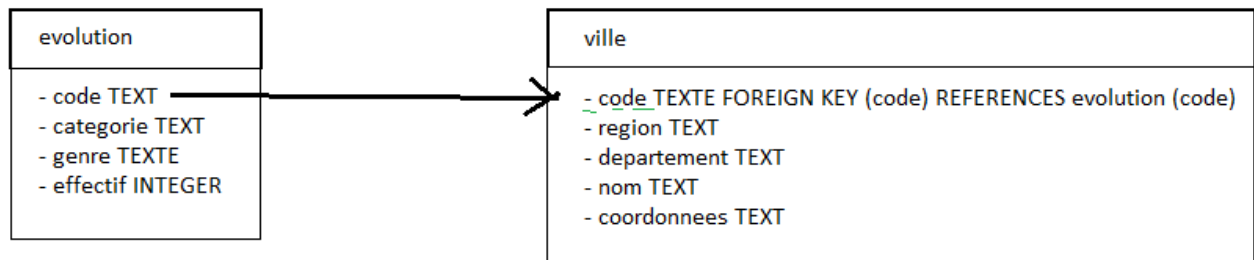
La table *evolution*:

- code TEXT
- categorie TEXT
- genre TEXTE
- effectif INTEGER

La table *ville*:

- code TEXTE FOREIGN KEY (code) REFERENCES evolution (code)
- region TEXT
- departement TEXT
- nom TEXT
- coordonnees TEXT

3.



4.

Pour paramétrer SQLite afin d'utiliser les contraintes d'intégrité référentielles, j'ai tapé les commandes suivantes:

```
PRAGMA foreign_keys = ON;
```

II. Sélection de données

1. Sélectionner toute la table evolution:

```
SELECT * FROM evolution;
```

2. Sélectionner les différentes catégories de la population du nord:

```
SELECT categorie FROM evolution;
```

3. Sélectionner les différentes catégories de la population du nord sans doublon:

```
SELECT DISTINCT categorie FROM evolution;
```

4. Sélectionner le nom et le code INSEE de la ville dont le code INSEE est 59350:

```
SELECT nom, code FROM ville WHERE code=59350;
```

5.

```
SELECT nom, code FROM ville WHERE nom='Caullery';
```

Le code INSEE de Caullery est 59140.

6. Sélectionner toutes les données de la table evolution de la ville dont le code INSEE est 59350:

```
SELECT * FROM evolution WHERE code=59350
```

7. Refaire cette sélection par ordre décroissant selon l'effectif:

```
SELECT * FROM evolution WHERE code=59350 ORDER BY effectif DESC;
```

8. les codes INSEE des villes ayant des catégories socioprofessionnelles dont les effectifs dépassent 2000 individus:

```
SELECT code FROM evolution WHERE effectif>2000 AND categorie!='Retraite';
```

9. Les villes ayant des catégories socioprofessionnelles dont les effectifs dépassent 2000 individus:

```
SELECT evolution.code, evolution.effectif, ville.nom FROM ville JOIN evolution ON  
ville.nom=evolution.categorie;
```

10. Même question mais sans doublons:

```
SELECT DISTINCT evolution.code, evolution.effectif, ville.nom FROM ville JOIN  
evolution ON ville.nom=evolution.categorie;
```

12.

```
SELECT SUM(effectif) FROM evolution WHERE genre='Femmes' AND  
categorie='Agriculteurs Exploitants';
```

Il y a 1911 femmes agricultrices dans le Nord.

13.